Noise-Conditioned Wavelet NAFNet for Blind Low-Light sRGB Denoising

Team Voxel — Mora SP Cup 2026, Preliminary Round

1 Problem and scoring objective

The task is blind denoising of 992×992 8-bit sRGB low-light photographs. Corruption strength varies per image and no noise-level metadata is available at inference. The official score is computed per image and averaged:

$$\text{composite} = 0.6\,\text{clip}\Big(\tfrac{\Delta\text{PSNR}}{15}, 0, 1\Big) + 0.4\,\max(\Delta\text{SSIM}, 0), \tag{1}$$

each delta measured against the noisy input. The $0.6/15 = 0.04$ gain per dB makes the metric PSNR-led while SSIM rewards structure preservation. Because the clip and the max are applied *before* averaging, per-image robustness matters more than mean performance. The model therefore predicts a residual at the original exposure and performs no gamma correction, tone mapping or global brightness adjustment.

2 Dataset observations

The 460 public noisy/clean pairs were characterised before any model was built. Residual variance is heteroscedastic and follows a Poisson–Gaussian law $\text{var}(I) = aI + b$, with normalised per-channel coefficients

$$a = (0.017095,\ 0.016532,\ 0.019045), \qquad b = (0.006362,\ 0.005684,\ 0.005492).$$

Three further observations drive the design. The read-noise floor b dominates dark regions, so corruption is strongest exactly where signal is weakest. Severity forms a *continuum* rather than discrete levels, which rules out a classifier over noise levels. And the residual is heavy-tailed with sparse impulses — 492 persistent hot pixels plus scattered defects — which a Gaussian model cannot represent. Together these motivate *spatial, content-dependent* conditioning rather than a single global noise scalar.

3 Classical signal processing

Three components operate directly on the noisy observation and carry **no learned parameters**. They supply the network with physical knowledge it would otherwise have to infer from data.

1. Analytical noise map. Evaluating $\sqrt{aI + b}$ with the fitted coefficients yields a per-pixel, per-channel standard-deviation estimate. This is the measured sensor model applied pointwise: the network is told how much corruption to expect at each intensity, instead of estimating it from a 256×256 patch where the estimate would be unreliable in dark or flat regions.

2. Exact 3×3 median via a sorting network. Impulse defects are not describable by a variance model, so they are detected separately. Nine replicate-padded neighbour tensors are sorted elementwise by a 36-comparison min/max bubble network; element four is the exact median. The channel-max absolute deviation from that median, scaled and clipped, becomes a one-channel impulse-confidence map. A sorting network is used rather than a library median because it is differentiable, backend-portable and exact — no histogram approximation.

3. Four-level orthonormal Haar pyramid. The LL band is recursively decomposed; at each scale the RGB LH, HL and HH details form nine channels at $H/2, \ldots, H/16$. This is a fixed filter bank, so the detail bands are a deterministic function of the input. Wavelet details expose high-frequency corruption at multiple scales *without* altering the RGB tensor the backbone consumes — the restoration path still sees the original observation.

Feeding these as *conditioning* rather than as extra input channels is deliberate: it leaves the pretrained backbone's input distribution untouched, so the wrapper can be initialised to an exact identity (Section 5).

4 Architecture

Backbone. NAFNet (width 32) with encoder depths $[2, 2, 4, 8]$, 12 middle blocks, decoder depths $[2, 2, 2, 2]$ and channel widths $32, 64, 128, 256, 512$. Each NAFBlock uses LayerNorm, pointwise expansion, depthwise 3×3 convolution, SimpleGate, simplified channel attention and two learned residual scales. PixelShuffle performs decoder upsampling; the network predicts a residual added to the reflect-padded input.

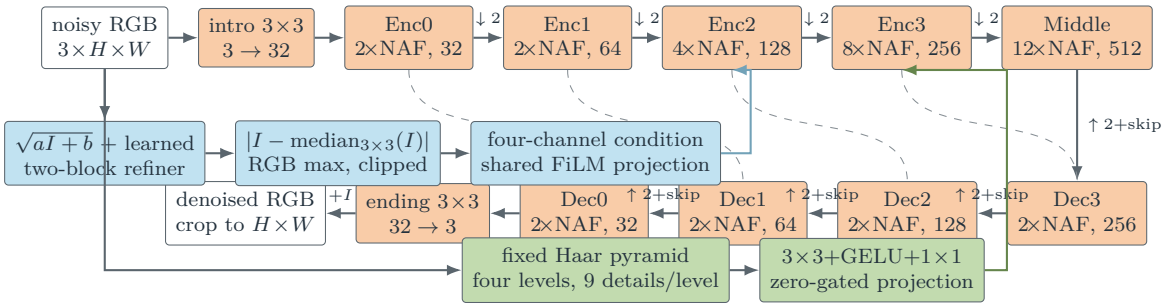


Figure 1: NCW-NAFNet. The NAFNet core is orange, noise conditioning blue, fixed-Haar detail injection green. Dashed lines are encoder–decoder skips.

Noise conditioning. The analytical branch computes $\sqrt{aI + b}$; a 16-channel, two-NAFBlock refiner predicts a tanh-bounded correction of at most ± 0.1, letting the model adapt the global sensor model to individual images. Concatenating the three sigma channels with the impulse channel gives a four-channel field C. At every encoder stage and at the middle block a shared 3×3 projection with two 1×1 heads generates FiLM parameters:

$$F' = (1 + g_\gamma \tanh \gamma(C)) \odot F + g_\beta \beta(C).$$

Conditioning is applied *before* the encoder output is stored as a skip, so the decoder also receives conditioned features.

Wavelet conditioning. At each scale a $3 \times 3 \to$ GELU $\to 1 \times 1$ adapter projects the nine Haar detail channels to the matching feature width and adds them after downsampling, through a zero-initialised scalar gate. Level ℓ injects at exactly the resolution produced by downsample ℓ, so no resampling occurs.

The model has 29,664,484 parameters: 29,159,715 in the backbone and 504,769 in the added modules — a 1.7 % parameter increase.

5 Identity safety

Every added path is scaled by a scalar gate initialised to zero, so before training the wrapper output is *numerically identical* to the backbone. This is a design guarantee,

not an approximation: the preflight check measured a maximum absolute difference of exactly 0.0, and the property is asserted by a unit test in `scripts/tests/test_model.py` . It means conditioning can only be adopted where it earns its place, and an untrained adapter can never destroy the restoration capability of the backbone.

Two repairs were needed on the DirectML backend: AdamW's in-place `lerp_` was replaced by an equivalent `addcmul_`/`addcdiv_` update, and whole-image validation was replaced by CPU-staged tiling. The official evaluator still scores complete 992×992 predictions.

6 Training protocol

Training ran in float32 on DirectML with batch size 1 and random 256×256 patches. Each sample was a real noisy/clean pair with probability 0.70; otherwise a clean patch was corrupted by a generalised synthesiser fitted to the measured statistics, with randomised a/b scaling, heavy-tail mixing and sparse defects. The objective was $L = 0.90L_{\text{PSNR}} + 0.10L_{\text{SSIM}}$ with gradient norm clipped at 1.0, $\beta = (0.9, 0.9)$ and weight decay 10^{-3}.

Phase	Interval	Trainable parameters	Backbone / adapter LR
A: adapter activation	steps 0–999	NCW additions only	$0 \,/\, 3.0 \times 10^{-4}$
B: targeted tuning	steps 1000–3424	middle, decoder, output + NCW	$3.0 \times 10^{-5} \,/\, 1.5 \times 10^{-4}$
C: low-LR refinement	steps 3425–4122	full model	cosine from $1.2 \times 10^{-5} \,/\, 7.0 \times 10^{-5}$

Table 1: Three phases. Phase C decayed to $1.8 \times 10^{-6}/1.05 \times 10^{-5}$ at the optimizer cutoff. Freezing the backbone during phase A lets the gates open before any pretrained weight moves.

Optimisation stopped at 52 minutes to reserve time for full validation and checkpoint selection, completing 4,122 steps and five validation epochs in 58.3 minutes.

7 Validation methodology and results

The 460 public pairs are split *deterministically and without RNG*: ids are ordered by blind noise estimate and every tenth is held out starting at index five, giving 414 training and 46 validation images spanning the full severity continuum ($\hat{\sigma}$ 7.57–40.66) rather than whichever end a random draw favoured. The split is regenerated on demand, so no stored state can drift. Image IDs 461–480 were excluded from all fitting, checkpoint selection and metric computation.

Every prediction is clipped, quantised round-half-up as $\lfloor 255x + 0.5 \rfloor$, written as PNG and scored by the *unmodified* `evaluation/evaluate.py`. The reported figures therefore include the exact 8-bit quantisation used for the submitted images.

The selected checkpoint achieves **31.3046 dB PSNR**, **0.919337 SSIM** and **0.63462481 composite** — roughly 2.4× the provided baseline's composite of ≈ 0.26, and a mean improvement of +11.49 dB PSNR and +0.4634 SSIM over the noisy inputs. Checkpoint selection used the official composite rather than PSNR: step 4122 trades 0.0029 dB for a 3.9×10^{-5} SSIM gain, which the 0.6/0.4 weighting rewards. Composite varies by only 0.00057 across the five epochs, indicating a stable, converged run rather than a lucky draw.

Epoch	Step	Phase	PSNR (dB)	SSIM	Composite
1	1000	A	31.3039	0.919125	0.634218
2	2000	B	31.2968	0.918920	0.634052
3	3000	B	31.3027	0.919209	0.634348
4	4000	C	31.3075	0.919298	0.634611
5	**4122**	**C**	**31.3046**	**0.919337**	**0.634625**
Provided starter baseline			—	—	≈ 0.26

Table 2: Validation trajectory over all 46 held-out images. Epoch 4 maximises PSNR; epoch 5 maximises the official composite and is the submitted checkpoint.

Released checkpoint. Weights ship in float16 (57 MB rather than 178 MB) and are upcast to float32 at load. Re-scored on CPU, the released file gives **31.3054 dB**, **0.919342**, **0.63464547** — $+2.0 \times 10^{-5}$ composite against the float32 original, with outputs differing by at most one 8-bit level on under 3 % of pixels.

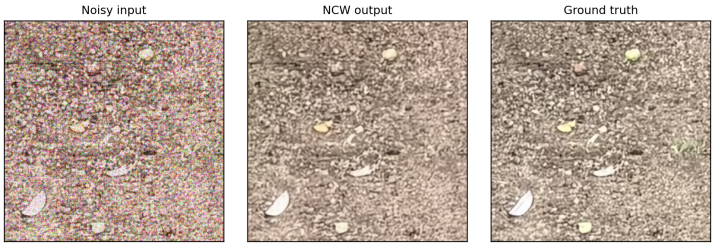


Figure 2: Centre crops for validation image 183, the most severe held-out image by blind noise estimate ($\hat{\sigma} = 40.66$ in 8-bit units). Strong noise is removed while the original exposure is retained.

Alternatives considered. Feeding the conditioning maps as additional input channels was rejected because it changes the backbone's input distribution and forecloses identity initialisation. Predicting a global noise scalar was rejected because measured severity is a continuum with strong spatial variation. Gamma correction and tone mapping were rejected outright: the metric compares against ground truth at the original exposure, so any brightness change is a direct PSNR loss.

8 Reproducibility

Inference is deterministic, CPU-capable and offline, using 256-pixel tiles with a 64-pixel discarded context halo and a raised-cosine blend over a 64-pixel overlap. The checkpoint embeds its own resolved configuration and metrics, so the model is rebuilt from the file itself and loaded with `strict=True` over all 766 tensors.

```
pythonscripts/denoise.py--noise_dir<in>--denoised_dir<out>
pythonscripts/validate.py
```

Submitted checkpoint `scripts/weights/ncw_1h_best_composite_fp16.pth`

SHA-256 `be1515a1afe066bbd445b6a5b73f04d0141d3ed08e22eaa4cb9fd145df86155b`

Backbone reference: S. Chen et al., "Simple Baselines for Image Restoration," ECCV 2022 (MIT licence, retained in `scripts/licenses/`). All metrics in this report are produced by the organizers' unmodified evaluator; no value is extrapolated from training loss.